

Classificador de Grãos de Soja

Migração do modelo: EfficientNet-B0 → YOLO11s-cls · Demo FATEC · 14/06/2026

1. O que mudou (resumo)

O classificador de produção foi trocado de **EfficientNet-B0 (TensorFlow/Keras)** para **YOLO11s-cls (Ultralytics/PyTorch)**. A decisão foi tomada após um experimento controlado que mediu os dois modelos nas **mesmas 57 fotos reais**, mesmo recorte, mesmo split de validação e augmentation equivalente. O YOLO venceu no domínio real (fotos de celular): **91,7%** contra **75%** do EfficientNet em igualdade de condições.

2. Resultado do experimento (val no domínio real, 12 fotos)

Modelo	Val real	Gap treino-val
EfficientNet — aug branda + mix Roboflow (antigo)	64,0%	25,0% (overfitting)
EfficientNet — aug forte + freeze parcial (justo)	75,0%	7,2%
YOLO11s-cls — aug forte (PRODUÇÃO)	91,7%	3,9%

Leitura: a **receita** de treino explica ~11pp (64→75) — a run antiga do EfficientNet usava learning rate baixo demais (1e-5), misturava 60% de dados do Roboflow (diluindo a adaptação) e media num holdout diferente. A **arquitetura** explica outros ~17pp (75→91,7): mesmo com tudo igualado, o YOLO venceu.

3. Por que o YOLO generaliza melhor com poucas fotos

- **AdamW + weight decay:** o Ultralytics auto-seleciona AdamW com regularização no otimizador; o EfficientNet rodava Adam puro. Com 57 fotos, weight decay é o principal freio de overfitting.
- **EMA (média móvel dos pesos):** o pipeline Ultralytics aplica EMA durante o treino, suavizando o checkpoint final e melhorando a generalização — o Keras não faz isso por padrão.
- **Atenção espacial (C2PSA):** o último bloco do YOLO11s usa self-attention sobre o mapa de features, ajudando a focar na textura/cor do grão e ignorar o fundo — exatamente o domain shift.

4. O gargalo real: domain shift

O modelo treina com fotos de **fundo preto** (dataset Roboflow). As fotos do celular têm **fundo cinza/metálico** e luz variada. Sem fine-tuning, o modelo lê o fundo como defeito e a acurácia despenca para ~3-8% (abaixo do acaso). O **fine-tuning no domínio real** (freeze parcial + augmentation forte de brilho/cor) recupera para **91,7%**. Por isso o fine-tuning deixou de ser opcional e virou etapa obrigatória do pipeline.

5. Mudanças no repositório

Arquivo	Mudança
model/app.py	Carrega .pt do YOLO (ultralytics) em vez de .keras; _predict reescrito
model/requirements.txt	ultralytics==8.4.67 substitui tensorflow==2.21.0
model/train_yolo.ipynb	NOVO — treino YOLO11s-cls no Roboflow (balanceado por oversampling)
model/finetune_yolo.ipynb	NOVO — fine-tuning no domínio real, self-contained (FT-0 bootstrap)

model/finetune_eff_fair.ipynb	NOVO — EfficientNet em condições idênticas (prova do experimento)
CLAUDE.md	Stack, decisões e §9 atualizados para refletir a troca

Núcleo da troca em `app.py` (a inferência passa um `PIL.Image RGB`, o mesmo caminho usado na validação que deu 91,7%):

```
model = YOLO("soja_yolo11s_finetuned.pt")

def _predict(crop):
    result = model.predict(Image.fromarray(crop), imgsz=224, verbose=False)[0]
    pv = result.probs.data.cpu().numpy() # reordenado p/ CLASS_NAMES
    return int(pv.argmax()), float(pv.max()), pv
```

6. Deploy no Hugging Face Space

- Subir `soja_yolo11s_finetuned.pt` (gerado pelo `finetune_yolo.ipynb`, FT-5) na raiz do Space
- Subir `app.py` e `requirements.txt` atualizados
- `soja_classes.json` permanece igual (mesma ordem de 5 classes)

Observação metodológica honesta: o conjunto de validação tem apenas 12 fotos, então 91,7% equivale a 11/12 — é uma tendência forte, não uma métrica de precisão estatística. O único erro do YOLO foi confundir `skin-damaged` com `broken`, a fronteira mais ambígua do dataset (ambos têm dano de superfície).